

PROJECT PART 1

GitHub Repository:

<https://github.com/rortizca/CSE-464-2026-rortizca>

Links to each commit:

- Feature 1:
<https://github.com/rortizca/CSE-464-2026-rortizca/commit/a763feed8abd8b9465a557530427a57c95f1ff6>
- Feature 2:
<https://github.com/rortizca/CSE-464-2026-rortizca/commit/733f29749bcad89ce9190dd48985d6e6eec0d4e7>
- Feature 3:
<https://github.com/rortizca/CSE-464-2026-rortizca/commit/68575452cc7001cbade2887fd60031bc87df7bf4>
- Feature 4:
<https://github.com/rortizca/CSE-464-2026-rortizca/commit/a7c7dd2f4b69898b560d97810f9883d622c36302>
- Features Merge:
<https://github.com/rortizca/CSE-464-2026-rortizca/commit/93d99135f7f73af625de8eb3015fe5c229e8c13>
- Remove node/edges feature commit:
<https://github.com/rortizca/CSE-464-2026-rortizca/commit/a98c8e03cd44a7edc2e6acba76a74f2669f3332e>
- Remove node/edges feature merge:
<https://github.com/rortizca/CSE-464-2026-rortizca/commit/754370b48a4d215ce1070eda40c57cff6a1423a5>
- Added Continuous Integration commit:
<https://github.com/rortizca/CSE-464-2026-rortizca/commit/b8e198405c9e1467c6387d39fe705285182cb53c>
- Added Continuous Integration merge:
<https://github.com/rortizca/CSE-464-2026-rortizca/commit/d0b4c2023b4bfc0501b8248507c24ad1a71bf0dc>

Instructions

The collection includes a dedicated Java Class object “Graph.java”, which contains all the API’s to create and modify a DOT graph file.

Each function operates according to the specification listed in the assignment description.

- 1) Start by creating a Graph object using the command below

```
Graph test_graph = new Graph();
```

- 2) Feature 1: You can parse a specified DOT file by utilizing the function parseGraph(String filepath)

```
test_graph.parseGraph("example.dot");
```

Input Graph:

```
digraph {  
  A -> B  
  B -> C  
  C -> A  
}
```

Expected Output:

```
Number of nodes: 3  
Nodes:  
A  
C  
B  
Edges:  
A -> B  
C -> A  
B -> C  
Number of Edges: 3
```

You can also output the graph to a string using the toString() function

```
String output = test_graph.toString();
```

As well, you could output the graph to a specified filename using the outputGraph(String filepath) function

```
test_graph.outputGraph("./outputPath");
```

- 3) Feature 2: You can add any number Node to any existing Graph by utilizing either the addNode(String label) or addNodes(String[] labels) functions

```
test_graph.addNode("A");
```

Shared input:

```
digraph {  
  A -> B  
}
```

Expected Output:

```
Number of nodes: 3
Nodes:
B
A
C
Edges:
A -> B
Number of Edges: 1
```

Or

```
String[] nodes = {"C", "D"};
test_graph.addNodes(nodes);
```

Expected Output:

```
Number of nodes: 4
Nodes:
B
A
D
C
Edges:
A -> B
Number of Edges: 1
```

- 4) Feature 3: You can add an edge between two distinct nodes using the function `addEdge(String srcLabel, String dstLabel)`

```
test_graph.addEdge("A", "B");
```

Input Graph:

```
digraph {
A
B
}
```

Expected Output:

```
Number of nodes: 2
Nodes:
B
A
Edges:
A -> B
Number of Edges: 1
```

- 5) Feature 4: You can output an imported Graph into a DOT file using `outputDOTGraph(String path)` or output that same Graph into a .png formatted file with a graphics output using `outputGraphics(String path, String format)`.

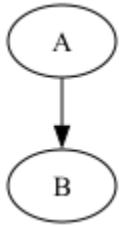
```
test_graph.outputDotGraph("./outputPath");
```

Or

```
test_graph.outputGraphics(outputPath, "png");
```

Input Graph:

```
digraph {  
A  
B  
A -> B  
}
```

Expected Output:

- 6) Remove Node Feature: You can remove a node from any graph by inputting the string name of a label into `removeNode(String label)`.

```
test_graph.removeNode("A");
```

Input Graph:

```
digraph {  
A  
B  
}
```

Output Graph:

```
digraph {  
B  
}
```

- 7) Remove Nodes Feature: You can remove an array of nodes from any graph by inputting the string array of labels into `removeNodes(String[] labels)`.

```
test_graph.removeNodes(new String[]{"A", "C"});
```

Input Graph:

```
digraph {  
A  
B  
C  
}
```

Output Graph:

```
digraph {  
B  
}
```

- 8) Remove Edge feature: You can remove an edge from any graph by inputting a source and destination node label into `removeEdge(String srcLabel, String dstLabel)`

```
9) test_graph.removeEdge("A", "C");
```

Input Graph:

```
digraph {  
A  
B  
C  
A -> C  
}
```

Output Graph:

```
digraph {  
A  
B  
C  
}
```

9) DFS or BFS algorithm Feature: You can search for the quickest path using either breadth first search or depth first search by inputting a source and destination node and an enum value: “BFS” or “DFS”

```
10) test_graph.removeEdge("A", "C", BFS);
```

```
test_graph.removeEdge("A", "C", DFS);
```